Лекция 4 · Алгоритмизация и программирование

Понятие алгоритма

Виды алгоритмов · Способы представления · Блок-схемы · UML-диаграммы

СПбГУПТД · ИИТА · 1 курс · 2 академических часа

План лекции

01

Алгоритм и его свойства

Определение, исполнитель, СКИ, шесть свойств, проблема остановки

02

Виды алгоритмов

Линейный, ветвящийся, циклический, вспомогательный и другие классификации

03

Способы записи

Словесный, псевдокод, графический, код, таблицы решений

04

Блок-схемы

ГОСТ 19.701-90, символы, правила построения, разбор примеров, типичные ошибки

05

UML

Четырнадцать диаграмм, диаграмма деятельности, сравнение с блок-схемой, инструменты

Зачем это нужно

Программист тратит на обдумывание решения больше времени, чем на набор кода. Язык программирования — последний шаг.

Как обычно делают

Открыть редактор и начать писать `for i in range(n)`, не представляя, что именно перебираешь.

Результат: код «почти работает», бесконечные циклы, невозможность объяснить своё решение на защите.

Как надо

Свести задачу к конечной последовательности точных действий.

Записать её так, чтобы понял человек: схемой, псевдокодом, диаграммой.

И только потом перевести на язык программирования.

Определение

Алгоритм — конечная последовательность точно определённых действий, которая по допустимым исходным данным за конечное число шагов приводит к результату.

От имени персидского математика IX века аль-Хорезми. Латинский перевод его трактата начинался словами «Dixit Algorizmi».

Три формализации 1930-х

Строгое определение понадобилось, когда потребовалось доказывать, что некоторые задачи алгоритмически неразрешимы. Три независимых модели оказались эквивалентны по вычислительной мощности.

Модель	Автор, год	Идея
Машина Тьюринга	А. Тьюринг, 1936	Головка, лента, таблица переходов
Лямбда-исчисление	А. Чёрч, 1936	Функции и их применение
Нормальные алгорифмы	А. А. Марков, 1951	Подстановки в строках

Тезис Чёрча — Тьюринга. Любой алгоритм в интуитивном смысле реализуем на машине Тьюринга. Это не теорема, а соглашение о том, что считать алгоритмом.

Исполнитель и его система команд

Алгоритм всегда пишется для конкретного исполнителя. Исполнитель характеризуется **СКИ** — набором команд, которые он понимает, и средой, в которой работает.

Студент

«Сложи числа в столбик»

Python

`sum(a)`

Процессор

`MOV, ADD, JMP`

Исполнитель формален

Он выполняет команды буквально и не догадывается о вашем замысле. «Посолить по вкусу» — команда для человека, но не для процессора.

Шесть свойств алгоритма

Дискретность

Разбит на отдельные завершённые шаги

Детерминированность

Каждый шаг определён однозначно, повтор даёт тот же результат

Понятность

Все команды входят в СКИ исполнителя

Конечность

Завершается за конечное число шагов

Результативность

По завершении есть результат или сообщение о невозможности

Массовость

Применим к классу входных данных, а не к одному набору

Проверяйте по этому списку любое своё решение до того, как открыть редактор.

Нарушения свойств

Инструкция	Какое свойство нарушено
«Плавно увеличивай скорость»	Дискретность
«Возьми любое число»	Детерминированность
«Соль по вкусу», «жарить до корочки»	Детерминированность, понятность
while True: без выхода	Конечность
«Реши уравнение 2x + 4 = 0»	Массовость

Отдельный случай — обращение к случайности: `random.randint(1, 6)` формально нарушает детерминированность. Такие алгоритмы выделяют в класс вероятностных.

Проблема остановки · А. Тьюринг, 1936

Не существует алгоритма, который для произвольной программы и произвольных входных данных определит, завершится она или зациклится.

Следствие для вас: конечность цикла проверить механически нельзя. За неё отвечает программист, а не компилятор и не IDE.

Три базовые конструкции

Теорема Бёма — Якопини, 1966. Любой алгоритм можно записать, используя только следование, ветвление и цикл. На этом стоит структурное программирование и запрет на goto.

01

Следование

Шаги выполняются один за другим. Линейный алгоритм.

02

Ветвление

По условию выполняется одна из ветвей. Разветвляющийся алгоритм.

03

Повторение

Группа шагов повторяется, пока выполняется условие. Циклический алгоритм.

Четвёртый вид — вспомогательный алгоритм (функция): не новая конструкция, а способ декомпозиции.

Линейный алгоритм

Шаги выполняются один за другим, ни один не пропускается и не повторяется. Порядок записи совпадает с порядком выполнения.

Словами

1. Ввести длину a
2. Ввести ширину b
3. Вычислить S = a · b
4. Вывести S

Python

```
a = float(input())
b = float(input())
s = a * b
print(s)
```

Почти любая реальная программа начинается с линейного участка: ввод данных и их подготовка.

Разветвляющийся алгоритм

В зависимости от условия выполняется одна из ветвей. Ветви обязательно сходятся: у конструкции один вход и один выход.

Неполное

`if`

Действие выполняется или пропускается

Полное

`if / else`

Выполняется ровно одна из двух ветвей

Множественный выбор

`elif / match`

Одна ветвь из нескольких

```
if x >= 0:
    y = x ** 0.5
else:
    print("Корень из отрицательного числа не определён")
```

Циклический алгоритм

Тело цикла, условие продолжения и параметр — три вещи, которые нужно назвать до того, как писать код.

Вид цикла	Когда проверяется условие	Python	Go
С предусловием	До тела; тело может не выполниться ни разу	`while cond:`	`for cond { }`
С постусловием	После тела; тело выполнится минимум один раз	`while True` + `break`	`for { … break }`
С параметром	Число повторений известно заранее	`for i in range(n):`	`for i := 0; i < n; i++`

В Go одно ключевое слово `for` покрывает все три случая, в Python отдельного цикла с постусловием нет — его эмулируют.

Вспомогательный и рекурсивный

Вспомогательный

Самостоятельный алгоритм, вызываемый из другого по имени. Даёт декомпозицию: большая задача разбивается на подзадачи, каждая решается и проверяется отдельно.

В блок-схеме обозначается символом «предопределённый процесс».

Рекурсивный

Вызывает сам себя на меньших данных. Обязательны база рекурсии и шаг, уменьшающий задачу, иначе нарушается конечность.

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

На практике любая программа — комбинация всех четырёх видов: линейный ввод, ветвление на проверке данных, цикл обработки, вызовы функций.

Другие классификации

По характеру задачи

- Вычислительные
- Поиск
- Сортировка
- Алгоритмы на графах
- Обработка данных
- Управляющие

По предсказуемости

- Детерминированные
- Вероятностные: Монте-Карло, тест Миллера — Рабина
- Точные
- Приближённые и эвристические: жадные, генетические, машинное обучение

По организации

- Последовательные
- Параллельные: несколько ядер, синхронизация, гонки данных
- Распределённые: узлы сети, обмен сообщениями

Параллельность нельзя выразить классической блок-схемой: у неё один поток управления. Именно здесь начинает работать UML.

Шесть способов записи

Способ	Кому адресован	Плюс	Минус
Словесный	Человеку, первичное описание	Не нужна подготовка	Многословен, двусмыслен
Формульно-словесный	В расчётных работах	Компактен в вычислениях	Плохо выражает управление
Псевдокод	Программисту	Близок к коду, не привязан к языку	Нет единого стандарта
Графический	Команде, заказчику, в документацию	Виден поток управления целиком	Громоздок для больших задач
Программный код	Исполнителю-машине	Единственный исполнимый вариант	Требует знания языка
Таблица решений	Аналитику, бизнес-логика	Полный перебор комбинаций условий	Не выражает циклы

Одна задача, три записи

Найти наибольшее из двух чисел.

Словесно

Ввести два числа a и b. Если a больше b, вывести a, иначе вывести b.

Псевдокод

```
НАЧАЛО
  ВВОД a, b
  ЕСЛИ a > b ТО
    ВЫВОД a
  ИНАЧЕ
    ВЫВОД b
  КОНЕЦ ЕСЛИ
КОНЕЦ
```

Python

```
a = int(input())
b = int(input())
print(a if a > b else b)
```

Словесная запись длиннее кода и при этом менее точна. Для двух чисел это терпимо, для алгоритма из сотни шагов — катастрофа. Отсюда и появились графические нотации.

Часть 4

Блок-схемы

Графическое представление алгоритма: шаги изображены геометрическими фигурами, порядок выполнения — линиями со стрелками.

ГОСТ 19.701-90

ЕСПД. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения. Полностью соответствует международному ISO 5807:1985. Пришёл на смену ГОСТ 19.002-80 и ГОСТ 19.003-80.

Основные символы



Терминатор

Начало и конец алгоритма



Процесс

Вычисление, присваивание, обработка



Решение

Проверка условия, два выхода



Данные

Ввод и вывод



Подготовка

Заголовок цикла с параметром



Подпрограмма

Вызов вспомогательного алгоритма



Граница цикла

Начало и конец цикла



Соединитель

Разрыв линии на странице

Ещё в стандарте: документ, межстраничный соединитель, комментарий, ручной ввод, дисплей

Правила построения

Оформление по ГОСТ

- Сторона a кратна 5 мм, высота b = 1,5a; размеры одинаковы по всей схеме
- Основное направление потока — сверху вниз и слева направо
- Линии вверх или справа налево — обязательно со стрелками
- Пересечения под прямым углом, слияние обозначается точкой
- Выходы решения подписаны: «да» / «нет»
- Символы нумеруются для ссылок из текста

Принципы

- Одна схема — один алгоритм или одна подпрограмма
- У каждой конструкции один вход и один выход, ветви сходятся
- В блоке — действие, а не строка кода на Python
- Уровень детализации одинаков по всей схеме
- Не умещается на страницу — выделяйте подпрограмму

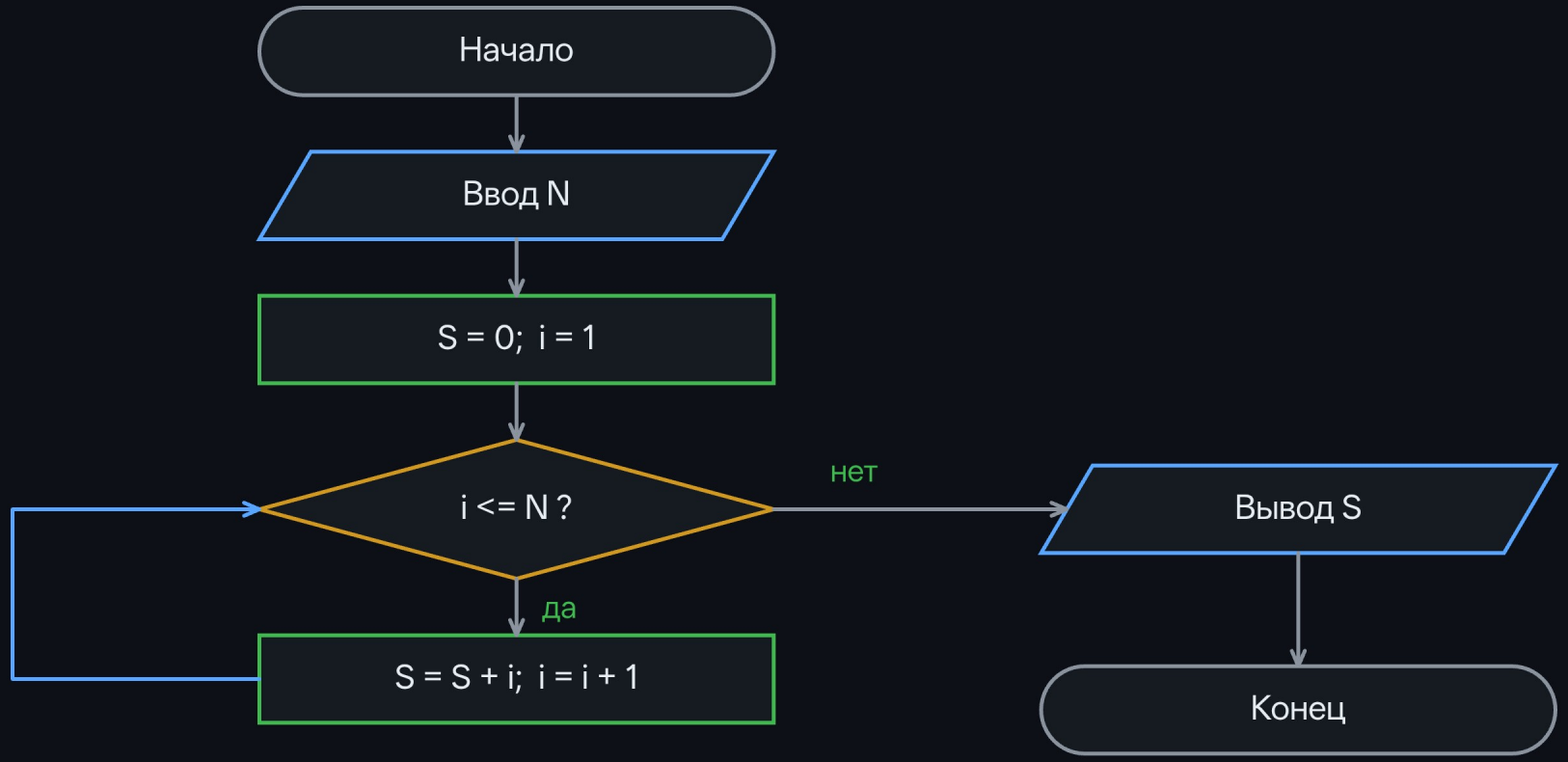
Проверка готовой схемы — трассировка: пройдите по ней вручную с конкретными числами, ведя таблицу значений переменных по шагам.

Ветвление: наибольшее из двух чисел



Обе ветви сходятся в блоке вывода — у конструкции один вход и один выход, терминатор «конец» один

Цикл: сумма чисел от 1 до N



Синяя линия — обратная связь цикла: она идёт вверх, поэтому стрелка на ней обязательна

Ошибки на защите лабораторных

Ошибка	Почему плохо	Как правильно
Ромб с одним выходом	Условие ничего не решает	Два выхода: «да» и «нет»
Выходы ромба не подписаны	Схема читается двояко	Подписать каждую ветвь
Несколько блоков «конец»	Нарушен принцип одного выхода	Свести ветви в один терминатор
Ввод-вывод в прямоугольнике	Смешаны обработка и обмен данными	Параллелограмм
Стрелка входит в блок сбоку или снизу	Направление потока неочевидно	Вход сверху, выход снизу
Цикл без изменения параметра	Бесконечный цикл	Блок изменения счётчика в теле
Условие записано как процесс	Потерян смысл проверки	Ромб или блок границы цикла

Проверка перед сдачей: трассировка схемы с конкретными числами и таблицей значений переменных

Часть 5

UML

Unified Modeling Language — стандартизированный графический язык для описания, проектирования и документирования программных систем.

Блок-схема описывает один алгоритм. UML описывает систему целиком: её структуру, поведение, взаимодействие и развёртывание.

1994–1997

Буч, Рамбо и Якобсон в Rational Software объединяют три нотации; UML 1.1 принят OMG

2005

UML 2.0: диаграмм становится 13, позже 14

2017

UML 2.5.1 — текущая версия; стандарт ISO/IEC 19505

Четырнадцать типов диаграмм

Структурные

Что система есть

- Классов
- Объектов
- Компонентов
- Композитной структуры
- Развёртывания
- Пакетов
- Профилей

Поведенческие

Что система делает

- Вариантов использования
- Деятельности
- Состояний
- Последовательности
- Коммуникации
- Обзора взаимодействия
- Синхронизации

Последние четыре объединяют в группу диаграмм взаимодействия. На практике используют пять-шесть типов из четырнадцати.

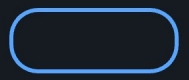
Диаграмма деятельности

Описывает поток работ или алгоритм: последовательность действий, ветвления, параллельные потоки. Ближайший родственник блок-схемы и её замена в проектной документации.



Начальный узел

Точка входа



Действие

Шаг работы любого масштаба



Решение и слияние

Условие в квадратных скобках



Fork и join

Запуск и ожидание параллельных потоков



Конечный узел

Завершение деятельности

Ещё два элемента: **дорожки** (swimlanes) показывают, кто выполняет действие, **объектные узлы** — какие данные передаются между действиями.

Пример: оформление заказа



После оплаты чек и посылка готовятся одновременно — классическая блок-схема так не умеет

Ещё четыре, которые пригодятся

Вариантов использования

Что система делает для пользователя. Актёры-человечки, овалы вариантов, отношения include и extend. С неё начинают проектирование: она фиксирует требования.

Классов

Прямоугольник из трёх секций: имя, атрибуты, методы. Видимость: + − # ~. Связи: ассоциация, агрегация, композиция, наследование, реализация.

Последовательности

Обмен сообщениями во времени: вертикальные линии жизни, время сверху вниз, сообщения — горизонтальные стрелки. Незаменима для связки клиент — сервер — база данных.

Состояний

Жизненный цикл объекта: состояния и переходы по событиям. Протоколы, экраны интерфейса, статусы заказа, игровые персонажи.

Деятельность отвечает на вопрос «в каком порядке выполняются действия», состояния — «в каком состоянии объект и что переводит его в другое».

Блок-схема или UML

Критерий	Блок-схема, ГОСТ 19.701-90	Диаграмма деятельности UML
Объект описания	Один алгоритм	Процесс, сценарий, алгоритм
Уровень детализации	Операторы программы	Действия любого масштаба
Параллельные потоки	Не выражаются	Узлы fork и join
Исполнители ролей	Не показываются	Дорожки
Стандарт	ГОСТ, ISO 5807	OMG UML 2.5.1, ISO/IEC 19505
Где требуют	Учебные работы, ЕСПД, ВКР	Проектная документация в индустрии

В пояснительной записке к ВКР нужны оба: блок-схемы ключевых алгоритмов в разделе реализации и UML-диаграммы в разделе проектирования.

Инструменты

draw.io

Бесплатный редактор в браузере, готовые наборы фигур ГОСТ и UML

yEd и MS Visio

Автоматическая укладка больших графов; Visio — стандарт корпоративной документации

PlantUML, Mermaid, Graphviz

Диаграмма из текста: лежит в репозитории рядом с кодом, правки видны в git diff, мышью двигать нечего

Mermaid

```
flowchart TD
  A([Начало]) --> B{n > 0 ?}
  B -- да --> C[Вывести n]
  B -- нет --> D[Вывести 0]
```

PlantUML

```
@startuml
start
if (n > 0 ?) then (да)
  :Вывести n;
else (нет)
  :Вывести 0;
endif
stop
@enduml
```

Оба текста превращаются в картинку одной командой или в онлайн-редакторе

Главное за лекцию

01

Алгоритм — конечная последовательность точных шагов для конкретного исполнителя, обладающая шестью свойствами

02

Любой алгоритм строится из трёх конструкций: следование, ветвление, цикл

03

Способ записи выбирается под читателя: псевдокод — себе, блок-схема — в отчёт, UML — команде, код — машине

04

Блок-схема регламентирована ГОСТ 19.701-90: один вход и один выход у конструкции, подписанные ветви решения, поток сверху вниз

05

UML описывает систему, а не один алгоритм; диаграмма деятельности умеет то, чего не умеет блок-схема: параллелизм и дорожки ответственности

Следующая лекция: линейные алгоритмы, типы данных и ввод-вывод

Задания к практике

01

Блок-схема решения квадратного уравнения: все случаи дискриминанта и вырожденный случай a = 0

02

Алгоритм Евклида: блок-схема, реализация на Python, сверка трассировки с работой программы на паре (48, 18)

03

Подсчёт чётных элементов массива тремя способами: словесно, псевдокодом, блок-схемой

04

Диаграмма деятельности «сдача лабораторной работы» в Mermaid или PlantUML с дорожками «студент» и «преподаватель»

Литература: ГОСТ 19.701-90 · Кормен и др., «Алгоритмы: построение и анализ» · Кнут, том 1, раздел 1.1 · Фаулер, «UML. Основы»